

SECTION A – CONCEPTUAL QUESTIONS

1. What is the difference between **SELECT *** and selecting specific columns?

In MySQL, **SELECT *** is used to retrieve all columns from a table.

Example:

```
SELECT * FROM orders;
```

This query displays all the columns and records from the orders table.

If we select specific columns, only the required columns are displayed.

Example:

```
SELECT  
    Order ID,  
    Sales,  
    Profit  
FROM orders;
```

This query displays only the Order ID, Sales, and Profit columns.

Difference:

- **SELECT *** retrieves all columns.
- Selecting specific columns retrieves only the required columns.

Selecting specific columns is useful because it makes the output easier to understand and avoids retrieving unnecessary information.

2. What is an alias in SQL? Explain the purpose of the **AS** keyword. Does an alias change the table structure?

An alias is a temporary name given to a column or table in a SQL query.

In MySQL, the **AS** keyword is used to create an alias.

Example:

```
SELECT Sales AS Total_Sales  
FROM orders;
```

In this example, the column Sales will be displayed as Total_Sales in the query result.

An alias does **not** change the actual table structure or the original column name. It only changes the name displayed in the result of that query.

For example, the actual column name remains Sales, even though the output shows Total_Sales.

3. Why can using quoted numeric values like '5000' cause type conflicts when filtering numeric columns?

In MySQL, quotation marks are generally used to represent text or string values.

For example:

'5000'

is treated as a string, whereas:

5000

is treated as a numeric value.

If the Sales column contains numeric values, it is better to use a numeric value without quotation marks.

Correct example:

```
SELECT *  
FROM orders  
WHERE Sales > 5000;
```

Using '5000' may cause MySQL to perform implicit type conversion because the value is written as a string.

Therefore, when filtering a numeric column, numeric values should normally be written without quotation marks.

4. In ORDER BY Profit DESC vs ASC, which one should be used to get the top 10 most profitable orders? Why?

To get the top 10 most profitable orders, we should use ORDER BY Profit DESC.

DESC means descending order, so the highest profit values are displayed first.

Example:

```
SELECT *  
FROM orders  
ORDER BY Profit DESC  
LIMIT 10;
```

This query sorts the orders from the highest profit to the lowest profit and then displays the first 10 records.

Meaning:

- DESC → Highest to lowest
- ASC → Lowest to highest

Therefore, DESC is used to find the top 10 most profitable orders.

5. What is the T-SQL equivalent of LIMIT 10 when using SQL Server?

This question is related to SQL Server, but since this assessment is being performed using MySQL, the MySQL syntax for limiting the result to 10 records is LIMIT 10.

Example:

```
SELECT *  
FROM orders  
LIMIT 10;
```

In MySQL, LIMIT is used to specify the maximum number of records that should be returned.

For example, to get the top 10 most profitable orders:

```
SELECT *  
FROM orders  
ORDER BY Profit DESC
```

LIMIT 10;

6. Explain the logical execution order of SELECT, WHERE, ORDER BY, and LIMIT.

In MySQL, the simplified logical execution order of these clauses is:

FROM → WHERE → SELECT → ORDER BY → LIMIT

For example:

```
SELECT `Order ID`, Sales, Profit
FROM orders
WHERE Sales > 5000
ORDER BY Profit DESC
LIMIT 10;
```

The query is logically processed as follows:

1. FROM

MySQL identifies the orders table from which the data will be retrieved.

2. WHERE

MySQL filters the records where Sales is greater than 5000.

3. SELECT

MySQL selects the required columns: Order ID, Sales, and Profit.

4. ORDER BY

The filtered records are sorted according to Profit in descending order.

5. LIMIT

Finally, MySQL returns only the first 10 records.

Therefore, the simplified logical execution order is:

FROM → WHERE → SELECT → ORDER BY → LIMIT